[52] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.

[53] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[54] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

[55] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.

[56] Zhihang Lin, Mingbao Lin, Yuan Xie, and Rongrong Ji. CPPO: Accelerating the training of group relative policy optimization-based reasoning models. *arXiv preprint arXiv:2503.22342*, 2025.

[57] Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

[58] Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Fei Huang, and Yan Zhang. ZeroSearch: Incentivize the search capability of llms without searching. *arXiv preprint arXiv:2505.04588*, 2025.

[59] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. ToolRL: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.

[60] Hongru Wang, Cheng Qian, Wanjun Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. OTC: Optimal tool calls via reinforcement learning. *arXiv preprint arXiv:2504.14870*, 2025.

[61] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[62] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. VinePPO: Unlocking rl potential for llm reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024.

[63] Xiangxiang Chu, Hailang Huang, Xiao Zhang, Fei Wei, and Yong Wang. GPG: A simple and strong reinforcement learning baseline for model reasoning. *arXiv preprint arXiv:2504.02546*, 2025.

[64] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.

[65] Max-Philipp B. Schrader. Gym-Sokoban. https://github.com/mpSchrader/gym-sokoban, 2018.

[66] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

A Source Code

As part of the new assets released with this work, we provide the complete source code at https://github.com/langfengQ/verl-agent. The repository includes a comprehensive `README.md` with installation instructions, training and evaluation procedures, and license information.

Our code is built upon the veRL framework [64] and extended with several features to support scalable training of long-horizon LLM agents. Key capabilities of our code include: (**1**) a step-wise multi-turn interaction paradigm that avoids concatenating the entire interaction history into a single input, as done in RAGEN [50], thereby ensuring scalability for very long-horizon optimization, (**2**) support for parallelized, gym-style group-based environments to enable high-throughput and state-aligned rollouts, (**3**) a rich suite of environments, including ALFWorld, WebShop, Sokoban, and Gym Cards; and (**4**) compatibility with both text-based and vision-language agents.

We provide a detailed installation guide (setup takes approximately five minutes), along with out-of-the-box scripts to reproduce our main results in ALFWorld and WebShop, as well as additional experiments on Sokoban and Gym Cards presented in Appendix D.3.

B Unbiasedness

We show that setting $F_{\text{norm}} = 1$ leads to an unbiased estimator up to a constant scaling factor. Our derivation follows the approach in [18]. For comparison, the standard REINFORCE Leave-One-Out (RLOO) [15, 16] is defined as:

$$A^{\text{RLOO}}(\tau_i) = R(\tau_i) - \frac{1}{N-1}\sum_{j\neq i} R(\tau_j). \tag{10}$$

We can relate $A^E(\tau_i)$ ($F_{\text{norm}} = 1$) and $A^{\text{RLOO}}(\tau_i)$ by introducing a scaling factor of $\frac{N}{N-1}$:

$$\begin{aligned}
\frac{N}{N-1}A^E(\tau_i) =& \frac{N}{N-1}R(\tau_i) - \frac{N}{N-1}\cdot\frac{1}{N}\sum_{j=1}^{N} R(\tau_j) && (11)\\
=& \frac{N}{N-1}R(\tau_i) - \frac{1}{N-1}R(\tau_i) - \frac{1}{N-1}\sum_{j\neq i} R(\tau_j) && (12)\\
=& R(\tau_i) - \frac{1}{N-1}\sum_{j\neq i} R(\tau_j) && (13)\\
=& A^{\text{RLOO}}(\tau_i) && (14)
\end{aligned}$$

Thus, setting $F_{\text{norm}} = 1$ corresponds to a rescaled version of $A^{\text{RLOO}}(\tau_i)$ and scaling the advantage by a constant does not affect the dynamics of policy gradient (it can be absorbed into the learning rate).

C Pseudo Code

Algorithm 1 summarizes the full GiGPO training procedure. Compared to vanilla GRPO, we highlight the additional parts introduced by GiGPO in italics. In particular, building step-level groups $G^S(\tilde{\boldsymbol{s}})$ is implemented by treating anchor states as keys and aggregating corresponding data into a hash table, which incurs minimal overhead. Furthermore, computing step relative advantages and combining advantages involve only simple arithmetic operations, both of which are lightweight. As such, GiGPO preserves the critic-free, low-memory, and stable convergence properties of group-based RL, while introducing fine-grained credit assignment that is particularly beneficial for training long-horizon LLM agents.

Algorithm 1 Training LLM Agents with GiGPO

Require: Initial policy $\pi_{\theta_{\text{old}}}$, task distribution $p(X)$, discount factor γ, weighting ω, clipping parameter ϵ, KL penalty β, group size N
for each training iteration **do**
  Update the old policy model: $\theta_{\text{old}} \leftarrow \theta$
  // Multi-step rollout phase
  Sample task $x \sim p(X)$ and initialize N identical environments
  for $t = 1$ to T **do**
    Sample actions $\left\{\boldsymbol{a}_t^{(i)} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \boldsymbol{s}_t^{(i)}, x)\right\}_{i=1}^{N}$
    Execute actions, observe rewards $\{r_t^{(i)}\}_{i=1}^{N}$ and next state $\{\boldsymbol{s}_{t+1}^{(i)}\}_{i=1}^{N}$
  end for
  // Grouping phase
  Compute episode relative advantages $A^E(\tau_i)$ via Equation (3)
  Build step-level groups $G^S(\tilde{\boldsymbol{s}})$ via the anchor states
  Compute step relative advantages $A^S(\boldsymbol{a}_t^{(i)})$ via Equation (7)
  // Policy update phase
  Combine advantages: $A(\boldsymbol{a}_t^{(i)}) = A^E(\tau_i) + \omega A^S(\boldsymbol{a}_t^{(i)})$
  Update policy θ by maximizing objective $\mathcal{J}_{\text{GiGPO}}(\theta)$
end for

D Experiment Details

D.1 Details of Training

Hyperparameters for ALFWorld. All methods are configured with identical hyperparameters: the maximum prompt length is 2048 tokens, and the maximum response length is 512 tokens. Each episode allows up to 50 environment steps. The learning rate is set to 1e-6 for the actor and 1e-5 for the critic (used only in PPO). We adopt a rule-based reward, assigning a reward of 10 for success and 0 for failure. To handle invalid actions generated by the agent, we apply a reward penalty of -0.1. For all group-based RL methods, we use a group size of 8 and sample 16 different groups per rollout, resulting in a total of $16 \times 8 = 128$ environments. In contrast, PPO uses 128 separate environments for rollouts. The rollout temperature is set to 1.0, while the validation temperature is set to 0.4. The mini-batch size is 256, and the KL-divergence loss coefficient is set to 0.01. For GiGPO, the weighting coefficient ω is fixed at 1 without further tuning, and the discount factor γ is set to 0.95.

Hyperparameters for WebShop. All methods are configured with identical hyperparameters: the maximum prompt length is 4096 tokens, and the maximum response length is 512 tokens. Each episode is limited to 15 environment steps. The learning rate is 1e-6 for the actor and 1e-5 for the critic (used only in PPO). We adopt a rule-based reward, assigning a reward of 10 for success and 0 for failure. Invalid actions are penalized with a reward of -0.1. As with ALFWorld, all group-based RL methods use a group size of 8 and sample 16 groups per rollout, totaling $16 \times 8 = 128$ environments. PPO, on the other hand, uses 128 distinct environments for rollouts. The rollout temperature is set to 1.0, while the validation temperature is set to 0.4. The mini-batch size is 64, and the KL-divergence loss coefficient is set to 0.01. For GiGPO, the weighting coefficient ω is set to 1 without additional tuning, and the discount factor γ is set to 0.95.

Computing Details. Experiments using Qwen2.5-1.5B-Instruct are conducted on two NVIDIA H100 GPUs, while those using Qwen2.5-7B-Instruct are trained on four NVIDIA H100 GPUs. Each experiment is trained for a total of 150 training iterations.

D.2 Prompts

The prompts we use for LLM agents are presented in Figure 7 and Figure 8. These prompt templates are constructed using Python-style string formatting, where placeholders enclosed in curly braces ({}) represent semantic slots. These placeholders, such as {task_description}, {step_count}, and {current_observation}, are dynamically populated at runtime via Python's `.format()` function. To enrich the agent's context, we use historical information and set the history length to 2. A fully instantiated example is shown in Figure 9 and Figure 10. This augmented representation is used as the final input to the agent.